Adrián Varela Jiménez, Diego Román Sánchez,
Adrián García García, Hugo Zafra Alarcos

PRÁCTICA 5 ALGORÍTMICA GRUPO A3 LIANA



UNIVERSIDAD
DE GRANADA

Adrián Varela Jiménez, Diego Román Sánchez,
Adrián García García, Hugo Zafra Alarcos

Índice

1. Reparto de raciones
    1.1. Descripción de la implementación
    1.2. Análisis de eficiencia
    1.3. Principio de optimalidad de Bellman
    1.4 Ejecución
2. El viaje del héroe
    2.1. Descripción de la implementación
    2.2. Análisis de eficiencia
    2.3. Principio de optimalidad de Bellman
    2.4 Ejecución

Reparto de raciones

Descripción de la implementación

El objetivo de este problema es determinar el número máximo de cajas para lograr la capacidad exacta M a partir de un conjunto de tipos de cajas con pesos y cantidades acotados.
La solución lleva a cabo el problema de optimización mediante una estrategia de Programación Dinámica descendente con memoización.
Se han creado un par de estructuras para diseñar la solución:

- Estado: Representa un subproblema y está indexado por una tupla formada por el índice de la caja (indice_caja) y la capacidad que queda por cubrir (peso_restante).
- Solución: Son aquellos datos que se deben devolver como solución. Incluye el número total de cajas (n_cajas), y el número de cajas utilizadas para cada tipo de caja (desglose_cajas). Cada tipo de caja se representa mediante un índice.

La ecuación de recurrencia empleada para este problema es:
Sol(i, cantidad) = max {k + Sol(i - i, cantidad - k * peso[i])}
Donde max busca el número de cajas más alto, y se llama recursivamente restando uno a i, y restando el peso del tipo de caja i por k, que representa el número de cajas elegidas de ese tipo.

Esta es la implementación del código:
La función reparto recibe por parámetro el índice del tipo de caja que se va a evaluar (i), la cantidad de kilogramos hasta llegar al límite (cantidad), la lista de cajas disponibles (cajas), es un vector de pair en el que el primer elemento representa el peso de esa caja y el segundo representa el número de cajas disponibles, por último, recibe la tabla de memoización (m), que es la estructura encargada de

almacenar soluciones de los subproblemas que el programa ya ha resuelto en llamadas anteriores. Está formado por un tipo Estado y un tipo Solución, de manera que si el estado actual ya ha sido comprobado, la solución se encuentra en tiempo logarítmico.

```
//Caso base éxito
if capacidad == 0 then
        return Solucion (n_cajas = 0, desglose = vector inicializado a 0);
end
//Caso base fracaso
if capacidad < 0 or i < 0 then
        return Solucion (n_cajas = -infinito, desglose = vector vacío
end
//Consulta de memoización
inicializar estado_actual
if estado_actual está en m then
        return m[estado_actual]
end
//Enumeración de decisiones secuenciales
inicializar peso_caja, mejor_solucion y disponibles
for k=0 hasta capacidad - k * peso_caja >= 0 and k <= disponibles do
        //Llamada recursiva
        sol_subproblemas = reparto (i - 1, capacidad - (k * peso_caja), cajas, m);
        //Si el camino de vuelta es factible, evaluamos optimalidad
        if sol_subproblemas != -infinito then
                candidatas_cajas = k + sol_subproblema * n_cajas;
                if candidata_cajas > mejor_solucion.n_cajas then
                        actualizamos todos los parámetros de mejor_solucion
                end
        end
end
//Guardar el estado actual y volver
m[estado_actual] = mejor_solucion;
return mejor_solucion;
```

Mostraremos la lógica de la solución implementada mediante un caso de uso:

- Peso objetivo = 7
- Cajas en el almacén
    - Tipo 0: Peso = 2, Cantidad = 2 cajas
    - Tipo 1: Peso = 3, Cantidad = 2 cajas

El algoritmo empieza invocando a la función reparto con los parámetros i = 1 y capacidad = 7.

- Para k = 0 (no se usa ninguna caja de 3 kg), se invoca a la función reparto con i = 0 y capacidad = 7. El bucle interno explora de k = 0 hasta k = 2, como ninguna combinación suma 7, devolverá -infinito, descartando esta rama por inviable.

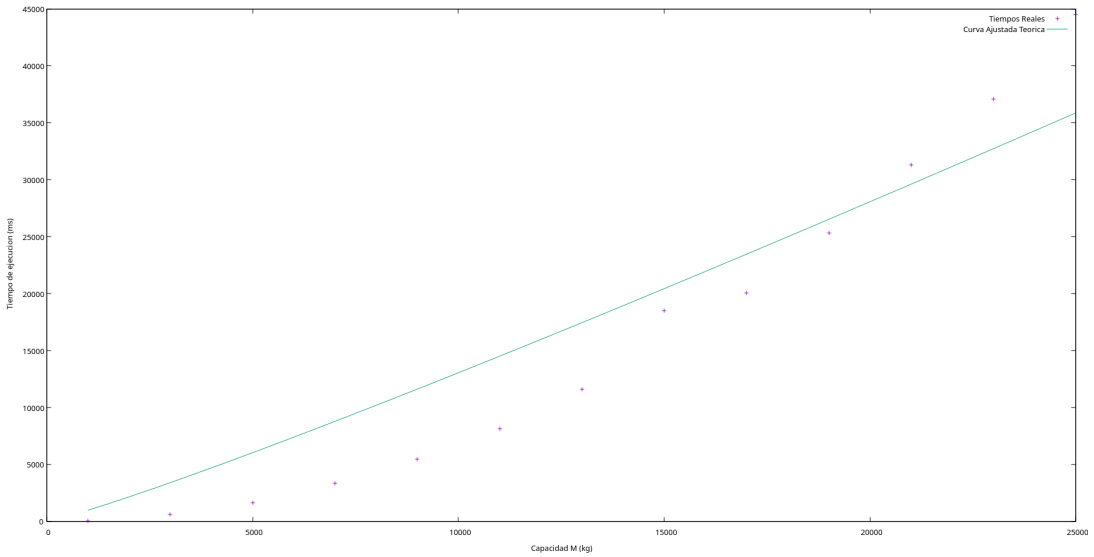
- Para k = 1, (se usa una caja de 3 kg), se invoca la función reparto con i = 0 y capacidad = 4. En el bucle interno, explorará la rama k = 2 (dos cajas de 2 kg), obteniendo una solución. Para k = 0 y k = 1 devolverá -infinito.
- Para k = 2 (se usan dos cajas de 3 kg), se invoca la función reparto con i = 0 y capacidad = 1. Para las cajas de tipo 0, con k = 0 no se llegará a la capacidad, y para k = 1 se superará la capacidad obtenida, devolviendo -infinito y podando la rama para que tampoco se pueda explorar k = 2.

La función principal en el estado original reparto(1, 7) compara los resultados devueltos por sus tres alternativas exploradas .El operador de selección determina que el máximo absoluto es 3 cajas. El algoritmo almacena el estado (1, 7) en el mapa de memoización asociado a esta solución definitiva y retorna con éxito al programa principal el desglose finalizado: 2 cajas del Tipo 0 y 1 caja del Tipo 1.

Análisis de eficiencia

Respecto a la eficiencia teórica, el algoritmo utiliza un enfoque descendente con memoización. El coste total viene determinado por el producto del número total de estados únicos por el coste de procesar cada estado.
Como un estado se define por una tupla, el parámetro i puede tomar valores de 0 hasta N - 1, y capacidad puede tomar valores de 0 hasta M, por tanto, el número de estados posibles es O(N*M).
Para un estado no calculado previamente, el algoritmo ejecuta un bucle for que itera según el número de cajas disponibles del tipo i. Si llamamos K al máximo número de cajas disponibles de cualquier tipo , el bucle realiza como máximo K+1 iteraciones.
Al utilizar un map, cada operación de búsqueda e inserción tiene un coste logarítmico respecto al número de elementos almacenados en el mapa. En el peor de los casos, este coste es O(log(N*M)).
Combinando estos factores, la eficiencia temporal sería de O(N*M*K*log(N*M)).

En cuanto a la eficiencia empírica, se han hecho ejecuciones para el mismo conjunto de cajas, pero aumentando la capacidad, y sin cambiar el tipo de cajas. Tras recolectar los resultados y representarlos a través de gnuplot, este ha sido el resultado:



Se puede observar un crecimiento positivo y sostenido a medida que crece M. A pesar de que la capacidad de multiplica, el tiempo de ejecución avanza prácticamente de forma lineal. Esto se debe a la memoización, que evita la degradación exponencial de rendimiento que se produce con la recursividad clásica.
Por tanto, presenta un comportamiento lineal logarítmico con respecto a M, con una tendencia que se acopla al orden O(Mlog(M)).

Además, se debe comentar la eficiencia espacial, en la que se guarda un máximo de N * M estados únicos. Esto se debe a la tabla que guarda los resultados (map), que guarda en su interior un objeto Solucion, que contiene un vector de enteros de tamaño N para recordar cuántas cajas se usaron de cada tipo. Al multiplicar los estados por el tamaño del vector interno, el coste se eleva a $O(N^2 * M)$.
También se debe sumar el coste de la pila de recursión, que en el peor caso será O(N).
Con esto, se puede concluir que la eficiencia espacial es de $O(N^2 * M)$.

Principio de optimalidad de Bellman

Para demostrar que este principio se cumple de forma estricta, realizaremos una prueba formal por contradicción.
Consideramos una instancia del problema donde deseamos alcanzar una capacidad exacta de M kilogramos utilizando N tipos de cajas disponibles.
Sea S = (k_{N-1}, k_{N-2}, …, k_0) la secuencia óptima de decisiones, donde cada k_i representa la cantidad de cajas elegidas del tipo i. El número total de cajas aportado por la solución es: B(S) = $\sum_{i=0}^{N-1} ki$.

Si evaluamos la primera etapa del problema (el tipo de caja N-1), se toma la decisión de emplear k_{N-1} cajas de dicho tipo. Esta decisión consume un peso total de k_{N-1}*peso[N-1], lo que descompone el problema original en un subproblema definido por disponer de los tipos de cajas restantes {N-2, N-1, …, 0} y por cubrir una capacidad restante de M’ = M - k_{N-1}*peso[N-1].

La subsecuencia de decisiones será S’ = (k_{N-2}, k_{N-3}, …, k_0) y el número de cajas será B(S’) = $\sum_{i=0}^{N-2} ki$.

Ahora, por reducción al absurdo, asumimos que el principio de Bellman no se cumple, lo que implicaría que la subsecuencia de decisiones restante S’ no es óptima para el subproblema de cubrir exactamente M’ kilogramos con los tipos de cajas del 0 al N-2.
Si S' no es óptima, significa que existe otra secuencia de decisiones alternativa y válida, S’’ = (k'_{N-2}, k'_{N-3}, …, k'_0), capaz de alcanzar la misma capacidad M' empleando los mismos tipos de cajas, pero aportando un número mayor de cajas.
Si esto es cierto, podríamos construir una nueva solución global para el problema original, S_{NUEVA}, combinando la primera decisión inicial con esta nueva subsecuencia:
S_{NUEVA} = (k_{N-1}, k'_{N-2}, k'_{N-3}, …, k'_0)
Ahora comprobamos el beneficio de esta nueva solución:

Peso Total = (k_{N-1}*peso[N-1]) + $\sum_{i=0}^{N-2}$ (k’i*peso[i]).

Como S’’ es una solución válida para el subproblema, sabemos que la sumatoria es igual a M’:
Peso Total = (k_{N-1}*peso[N-1]) + M’

Peso Total = (k_{N-1}*peso[N-1]) + (M - k_{N-1}*peso[N-1]) = M
Ahora verificamos la función objetivo:
$B(S_{NUEVA}) = k_{N-1} + B(S'')$
Como $B(S'') > B(S')$, entonces
$B(S_{NUEVA}) > k_{N-1} + B(S')$
Y como $k_{N-1} + B(S')$ es justo el número de cajas de nuestra solución óptima original $B(S)$, llegamos a que $B(S_{NUEVA}) > B(S)$.
Hemos obtenido una solución válida que utiliza más cajas que la solución S para el mismo peso M. Esto contradice nuestra hipótesis inicial de que S era la solución óptima absoluta.
Por lo tanto, la suposición de que la subsecuencia de decisiones restante S' no era óptima es falsa, demostrando que toda subsecuencia de decisiones de una solución óptima debe ser a su vez óptima para el subproblema que resuelve, cumpliéndose el Principio de Optimalidad de Bellman.

Ejecución

Para compilar el programa es suficiente con dirigirse al directorio src y escribir el comando g++ reparto_raciones.cpp -o ../bin/reparto
El ejecutable se encuentra en la carpeta bin. Para ejecutarlo desde el directorio src, se utiliza el comando ./../bin/reparto M, donde M representa el número de kilogramos a entregar.

Está diseñado de manera que los datos de ejecución se dan de manera aleatoria.
Para cada ejecución habrá entre 2 y 5 tipos de cajas, cuyos pesos irán de 2 a 10, y además, la cantidad de cajas disponibles estará entre 1 y 5 unidades.

Aquí hay varios ejemplos de ejecución:

- ./../bin/reparto 22:
  Se han generado los siguientes datos:

Tipo caja	**Peso**	**Cantidad**
0	5	2
1	2	4
2	6	3

  Para llegar al número máximo de cajas, el algoritmo nos devuelve que se deben coger 2 cajas del tipo 0, 3 del tipo 1 y 1 caja del tipo 2.
  Esto es: 2*5 + 3*2 + 1+6 = 22 kg
  Esta es la ejecución en la terminal:

```
Hugo~/P5_ALG/src$ ./../bin/reparto 22
Cajas Disponibles en el Almacen
Tipo 0 -> Peso: 5 kg | Cantidad: 2
Tipo 1 -> Peso: 2 kg | Cantidad: 4
Tipo 2 -> Peso: 6 kg | Cantidad: 3
--------------------------------------

Resultado para 22 kg
El maximo numero de cajas es: 6
Detalle del reparto óptimo:
- Del Tipo 0 (Peso: 5 kg) se utilizaron: 2 cajas.
- Del Tipo 1 (Peso: 2 kg) se utilizaron: 3 cajas.
- Del Tipo 2 (Peso: 6 kg) se utilizaron: 1 cajas.
```

- ./../bin/reparto 17
  Se han generado los siguientes datos:

Tipo caja	**Peso**	**Cantidad**
0	10	5
1	9	4
2	2	1

Con esta combinación de datos, no es posible llegar al peso requerido.

```
Hugo~/P5_ALG/src$ ./../bin/reparto 17
Cajas Disponibles en el Almacen
Tipo 0 -> Peso: 10 kg | Cantidad: 5
Tipo 1 -> Peso: 9 kg | Cantidad: 4
Tipo 2 -> Peso: 2 kg | Cantidad: 1
----------------------------------------

Resultado para 17 kg
No es posible entregar exactamente 17 kg.
```

Con todo esto, se puede verificar que el programa encuentra siempre el resultado óptimo ya que cumple con la propiedad de maximización, priorizando aquellas cajas menos pesadas antes de recurrir a los contenedores grandes.

Adrián Varela Jiménez, Diego Román Sánchez,
Adrián García García, Hugo Zafra Alarcos

EL viaje del héroe

Descripción de la implementación

La solución que hemos desarrollado se basa en el algoritmo de Programación Dinámica de Floyd-Warshall, pero siendo adaptado por nosotros para que cumpla con las restricciones de "El viaje del héroe". Este se diferencia del algoritmo original en que, mientras que el original solo suma distancias de las aristas, el nuestro incorpora el coste adicional de vencer a los guardianes de las localizaciones intermedias.

En la fase de inicialización lo único que hay que hacer es sustituir los valores de la matriz D (costes mínimos) con los de la matriz C (coste directo).

```
Para i desde 0 hasta n - 1 hacer
  Para j desde 0 hasta n - 1 hacer
    D[i][j] ← C[i][j]
    P[i][j] ← -1
  Fin Para
Fin Para
```

En la fase de recurrencia se establecen tres bucles, el exterior (k) para evaluar cada posible nodo intermedio, y el intermedio y final (**i** y **j**) como nodos de salida y llegada respectivamente. Estos se utilizan para calcular el coste de viajar de i a j pasando por k, y en el caso de que el coste sea menor que el guardado actualmente en D, se actualiza el valor en esta y se registra el nodo k en P para poder reconstruir la ruta más barata posteriormente.

```
Para k desde 0 hasta n - 1 hacer
    Para i desde 0 hasta n - 1 hacer
      Para j desde 0 hasta n - 1 hacer
        Si (D[i][k] + G[k] + D[k][j]) < D[i][j] entonces
          D[i][j] ← D[i][k] + G[k] + D[k][j]
          P[i][j] ← k
        Fin Si
      Fin Para
    Fin Para
Fin Para
```

Por último se **devuelven** las matrices D y P.

Ahora, para ilustrar el funcionamiento del algoritmo imaginemos un mapa sencillo de tres nodos:

Nodo 0: Castillo
Nodo 1: Bosque (con un guardián de coste G[1] = 5)
Nodo 2: Taberna

Supongamos además que los caminos directos (matriz C) son:

De 0 a 1: coste 10.
De 1 a 2: coste 20.
De 0 a 2: coste 50.

Paso 1: Inicialización

La matriz D se copia con los mismos valores de la matriz C. En este punto, el coste para ir desde el castillo hasta la taberna es 50 y la matriz P está llena de -1.

Paso 2: Evaluando k = 0

El algoritmo comprueba si pasando por el castillo mejora alguna ruta, y como no hay ninguna mejora el algoritmo prosigue.

Paso 3: Evaluando k = 1

El algoritmo evalúa todas las combinaciones posibles pasando por el bosque y llegamos al siguiente caso:

Coste actual D[0][2] = 50
Coste alternativo = D[0][1] (Castillo a Bosque) + G[1] (Vencer al guardián del Bosque) + D[1][2] (Bosque a Taberna).
Coste alternativo = 10 + 5 + 20 = 35.

Entonces, como 35 < 50, el algoritmo detecta que se puede optimizar el coste del recorrido por lo que actualiza el valor de D[0][2] a 35 y anota en P[0][2] el valor 1 indicando así que para ir de la casilla 0 a la 2 hay que pasar por la 1.

Paso 4: Evaluando k = 2

Se vuelven a evaluar las rutas posibles teniendo como casilla intermedia la 2 pero no se puede mejorar por lo que se termina el algoritmo.

Análisis de eficiencia

Eficiencia Teórica

- Complejidad Temporal

  El algoritmo consta de dos bucles principales, el primero que es la inicialización de las matrices, y el segundo que es la parte de recurrencia.

  El primero, al ser dos bucles anidados con N iteraciones cada uno para la inicialización de las matrices tiene orden $O(N^2)$.
  El segundo bloque consta de tres bucles anidados con N iteraciones cada uno al igual que el anterior bloque, por lo que este bloque tendrá orden $O(N^3)$.

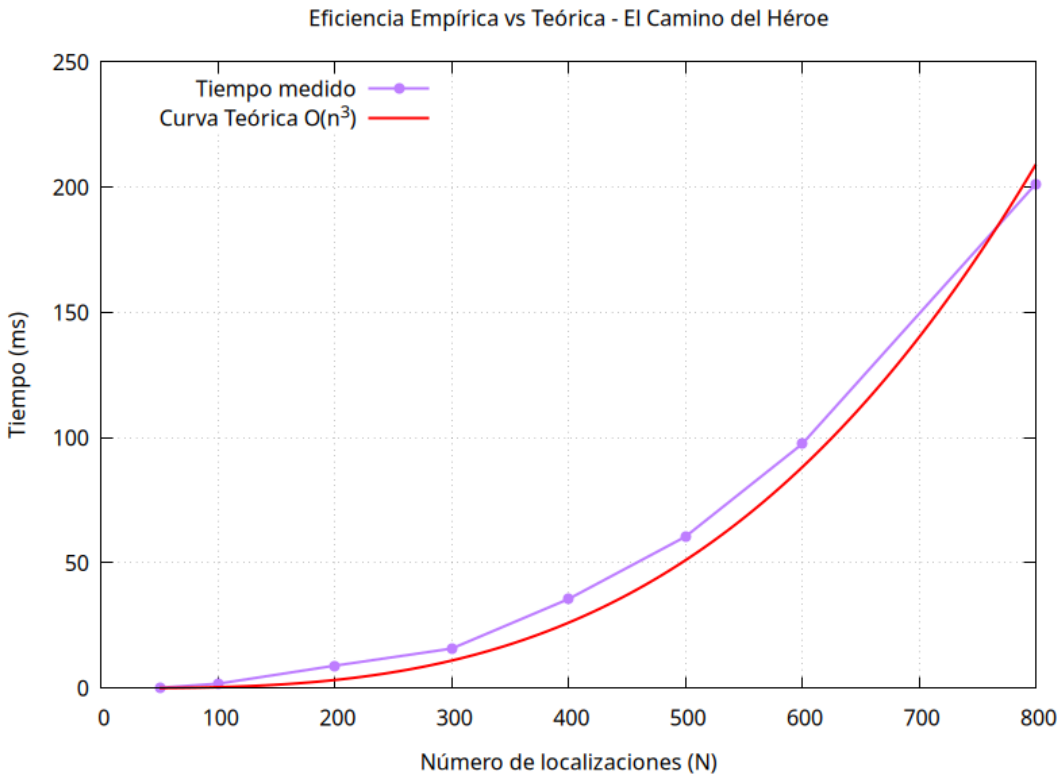
  Por tanto, el algoritmo se nos quedará con el siguiente orden de eficiencia: $O(N^2) + O(N^3) = O(N^3)$.

- Complejidad Espacial

  Para el algoritmo se utilizan dos matrices cuadradas con dimensiones NxN, además de las estructuras de entrada C y G. Por tanto, como el espacio de memoria requerido crece de forma cuadrática respecto al tamaño de N, se nos queda una eficiencia espacial de $O(N^2)$.

Eficiencia Empírica

Para contrastar el análisis teórico que hemos obtenido antes, usando el generador de mapas aleatorios, hemos hecho varias pruebas de ejecución con distintos tamaños desde 50 hasta 800.



En la gráfica que hemos generado, hemos comparado los tiempos empíricos con la curva teórica de $O(N^3)$ y obtenemos las siguientes conclusiones:

-Los datos empíricos crecen de manera casi idéntica que la curva teórica que hemos dibujado, por lo que prácticamente podemos afirmar que el crecimiento es cúbico.

-A pesar de que hay pequeñas desviaciones y se separan un poco ambas gráficas, esto puede deberse simplemente al hardware, ya que el análisis empírico depende directamente del mismo.

Adrián Varela Jiménez, Diego Román Sánchez,
Adrián García García, Hugo Zafra Alarcos

Principio de optimalidad de Bellman

El algoritmo implementado se basa en el principio de optimalidad de Bellman, que establece que toda subruta de una ruta óptima es a su vez óptima. Dicho de otro modo, si el camino de menor coste entre dos localizaciones i y j pasa por una localización intermedia k, entonces el tramo de i hasta k y el tramo de k hasta j son también los de menor coste posible entre esos pares de nodos.

En este problema,la recurrencia que expresa dicho principio es la siguiente:

D(k)[i][j] = min(D(k-1)[i][j] , D(k-1)[i][k] + G(k) + D(k-1)[k][j])

Donde D(k)[i][j] representa el coste mínimo para ir de i a j utilizando únicamente las localizaciones 1, 2, ..., k como posibles intermedias. G(k) es el coste del guardián de la localización k, que solo se paga cuando k actúa como nodo intermedio,y nunca cuando es origen ni destino del viaje.

El caso base es D(0)[i][j] = C[i][j], es decir el coste del camino directo entre i y j sin pasar por ningún intermedio, o infinito si dicho camino no existe. Sobre este caso base, el algoritmo va incorporando sucesivamente cada localización k como posible intermedio y va actualizando los costes si la nueva alternativa mejora el valor actual.

La validez del principio de optimalidad se justifica por contradicción ya que si existiera una subruta de la ruta óptima que no fuera óptima, se podría sustituir por una mejor reduciendo el coste total, lo que contradice que la ruta original era óptima. Dado que el coste de un camino en este problema es la suma de costes de tramos directos más los costes de los guardianes intermedios, y la suma es una operación que respeta la estructura de subproblemas, el principio se cumple y la programación dinámica produce una solución óptima global.

Ejecución

El proyecto tiene tres módulos fuente que deben compilarse juntos. Desde la raíz del proyecto ejecutamos los siguientes comandos:

```
g++ -o bin/aventura src/main.cpp src/camino_heroe.cpp src/visualizacion.cpp
g++ -o bin/generador src/generador.cpp
```

El primer comando produce el ejecutable principal aventura que lee un mapa, aplica el algoritmo e imprime los resultados. El segundo compila el generador de mapas aleatorios generador, utilizado para crear casos de prueba

EL ejecutable aventura tiene estas formas

./bin/aventura <entrada.txt>

./bin/aventura <entrada.txt> <salida.txt>

./bin/aventura <entrada.txt> <salida.txt> <mapa.svg>

./bin/aventura <entrada.txt> <salida.txt> <mapa.svg> <origen> <destino>

La última forma, que es la utilizada en los ejemplos de esta sección, genera además un mapa visual en formato SVG con la ruta óptima entre dos nodos

EJEMPLO 1:

Comando : ./bin/aventura data/Entradas/entrada_ejemplo1.txt

data/Salidas_Texto/ejemplo1.txt

data/Salidas_DIbujo/ejemplo1.svg

1 4

CASO DE USO 1:

4

0 3 8 -1

-1 0 5 2

-1 -1 0 1

7 -1 -1 0

4 2 1 3

MAPA

Localizaciones:

[1] guardian con coste G = 4

[2] guardian con coste G = 2

[3] guardian con coste G = 1

[4] guardian con coste G = 3

Los caminos directos son:

[1] --(3)--> [2]

[1] --(8)--> [3]

[2] --(5)--> [3]

[2] --(2)--> [4]

[3] --(1)--> [4]

[4] --(7)--> [1]

La matriz de costes mínimos:

	1	2	3	4
1:	0	3	8	7
2:	12	0	5	2
3:	11	18	0	1
4:	7	14	19	0

Las rutas óptimas:

1 -> 2: 1 -> 2 [C(1->2)=3] (coste 3)

1 -> 3: 1 -> 3 [C(1->3)=8] (coste 8)

1 -> 4: 1 -> 2 -> 4 [C(1->2)=3 + G(2)=2 + C(2->4)=2] (coste 7)

2 -> 1: 2 -> 4 -> 1 [C(2->4)=2 + G(4)=3 + C(4->1)=7] (coste 12)

2 -> 3: 2 -> 3 [C(2->3)=5] (coste 5)

2 -> 4: 2 -> 4 [C(2->4)=2] (coste 2)

3 -> 1: 3 -> 4 -> 1 [C(3->4)=1 + G(4)=3 + C(4->1)=7] (coste 11)

3 -> 2: 3 -> 4 -> 1 -> 2 [C(3->4)=1 + G(4)=3 + C(4->1)=7 + G(1)=4 + C(1->2)=3] (coste 18)

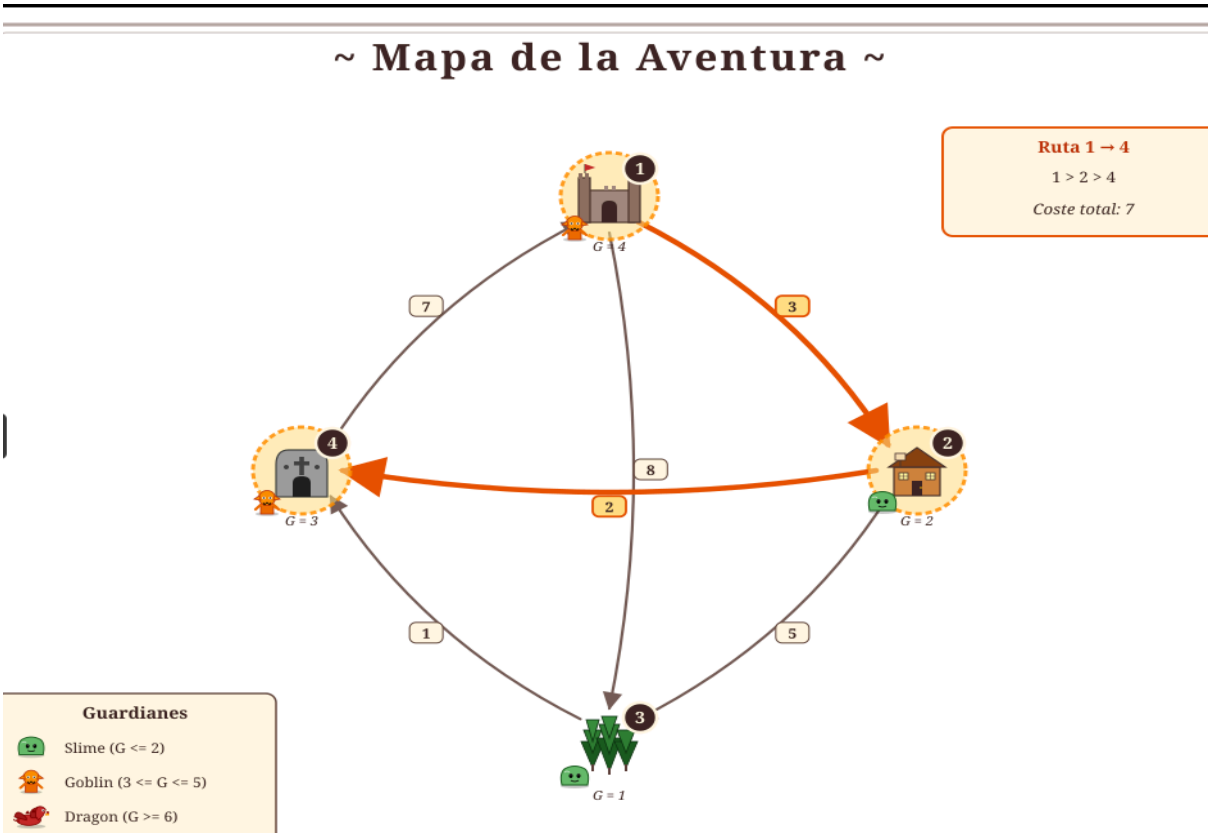
3 -> 4: 3 -> 4 [C(3->4)=1] (coste 1)

4 -> 1: 4 -> 1 [C(4->1)=7] (coste 7)

4 -> 2: 4 -> 1 -> 2 [C(4->1)=7 + G(1)=4 + C(1->2)=3] (coste 14)

4 -> 3: 4 -> 1 -> 3 [C(4->1)=7 + G(1)=4 + C(1->3)=8] (coste 19)

La ruta 1⟶4 nos revela el comportamiento del algoritmo. No existe un camino directo entre ambos nodos, por lo que se evalúan los intermedios posibles, pasar por el nodo 2 tiene coste C(1⟶2) + G(2) + C(2⟶4) = 3 + 2 + 2 = 7, mientras que pasar por el nodo 3 costaría C(1⟶3) + G(3) + C(3⟶4) = 8 + 1 + 1 = 10. El algoritmo selecciona correctamente el nodo 2 como intermedio, aunque el guardián del nodo 3 sea más barato, porque el trayecto total es menor.



EJEMPLO 2:

Comando: Comando : ./bin/aventura data/Entradas/entrada_ejemplo2.txt

data/Salidas_Texto/ejemplo2.txt

data/Salidas_DIbujo/ejemplo2.svg

2 6

Adrián Varela Jiménez, Diego Román Sánchez,
Adrián García García, Hugo Zafra Alarcos

CASO DE USO 2:

6

0 2 15 -1 -1 7

8 0 4 -1 -1 6

-1 -1 0 12 -1 7

-1 12 15 0 5 3

-1 -1 8 -1 0 7

15 -1 2 8 15 0

4 3 3 3 7 4

MAPA

Localizaciones:

[1] guardián con coste G = 4

[2] guardián con coste G = 3

[3] guardián con coste G = 3

[4] guardian con coste G = 3

[5] guardian con coste G = 7

[6] guardian con coste G = 4

Los caminos directos son:

[1] --(2)--> [2]

[1] --(15)--> [3]

[1] --(7)--> [6]

[2] --(8)--> [1]

[2] --(4)--> [3]

[2] --(6)--> [6]

[3] --(12)--> [4]

[3] --(7)--> [6]

[4] --(12)--> [2]

[4] --(15)--> [3]

[4] --(5)--> [5]

[4] --(3)--> [6]

[5] --(8)--> [3]

[5] --(7)--> [6]

[6] --(15)--> [1]

[6] --(2)--> [3]

[6] --(8)--> [4]

[6] --(15)--> [5]

La matriz de costes mínimos:

	1	2	3	4	5	6
1:	0	2	9	19	26	7
2:	8	0	4	18	25	6
3:	26	27	0	12	20	7
4:	22	12	9	0	5	3
5:	26	32	8	19	0	7

6: 15 21 2 8 15 0

Las rutas óptimas:

1 -> 2: 1 -> 2 [C(1->2)=2] (coste 2)

1 -> 3: 1 -> 2 -> 3 [C(1->2)=2 + G(2)=3 + C(2->3)=4] (coste 9)

1 -> 4: 1 -> 6 -> 4 [C(1->6)=7 + G(6)=4 + C(6->4)=8] (coste 19)

1 -> 5: 1 -> 6 -> 5 [C(1->6)=7 + G(6)=4 + C(6->5)=15] (coste 26)

1 -> 6: 1 -> 6 [C(1->6)=7] (coste 7)

2 -> 1: 2 -> 1 [C(2->1)=8] (coste 8)

2 -> 3: 2 -> 3 [C(2->3)=4] (coste 4)

2 -> 4: 2 -> 6 -> 4 [C(2->6)=6 + G(6)=4 + C(6->4)=8] (coste 18)

2 -> 5: 2 -> 6 -> 5 [C(2->6)=6 + G(6)=4 + C(6->5)=15] (coste 25)

2 -> 6: 2 -> 6 [C(2->6)=6] (coste 6)

3 -> 1: 3 -> 6 -> 1 [C(3->6)=7 + G(6)=4 + C(6->1)=15] (coste 26)

3 -> 2: 3 -> 4 -> 2 [C(3->4)=12 + G(4)=3 + C(4->2)=12] (coste 27)

3 -> 4: 3 -> 4 [C(3->4)=12] (coste 12)

3 -> 5: 3 -> 4 -> 5 [C(3->4)=12 + G(4)=3 + C(4->5)=5] (coste 20)

3 -> 6: 3 -> 6 [C(3->6)=7] (coste 7)

4 -> 1: 4 -> 6 -> 1 [C(4->6)=3 + G(6)=4 + C(6->1)=15] (coste 22)

4 -> 2: 4 -> 2 [C(4->2)=12] (coste 12)

4 -> 3: 4 -> 6 -> 3 [C(4->6)=3 + G(6)=4 + C(6->3)=2] (coste 9)

4 -> 5: 4 -> 5 [C(4->5)=5] (coste 5)

4 -> 6: 4 -> 6 [C(4->6)=3] (coste 3)

5 -> 1: 5 -> 6 -> 1 [C(5->6)=7 + G(6)=4 + C(6->1)=15] (coste 26)

5 -> 2: 5 -> 6 -> 1 -> 2 [C(5->6)=7 + G(6)=4 + C(6->1)=15 + G(1)=4 + C(1->2)=2] (coste 32)

5 -> 3: 5 -> 3 [C(5->3)=8] (coste 8)

5 -> 4: 5 -> 6 -> 4 [C(5->6)=7 + G(6)=4 + C(6->4)=8] (coste 19)

5 -> 6: 5 -> 6 [C(5->6)=7] (coste 7)

6 -> 1: 6 -> 1 [C(6->1)=15] (coste 15)

6 -> 2: 6 -> 1 -> 2 [C(6->1)=15 + G(1)=4 + C(1->2)=2] (coste 21)

6 -> 3: 6 -> 3 [C(6->3)=2] (coste 2)

6 -> 4: 6 -> 4 [C(6->4)=8] (coste 8)

6 -> 5: 6 -> 5 [C(6->5)=15] (coste 15)

El mapa tiene 6 localizaciones. Los costes de los guardianes son G(1)=4, G(2)=3, G(3)=3, G(4)=3, G(5)=7, G(6)=4. El guardián del nodo 5 es el más caro (G=7), por lo que el algoritmo lo evita como intermedio siempre que sea posible. El nodo 6 actúa como hub central al tener conexiones directas baratas hacia varios nodos (C(6→3)=2, C(6→4)=8).

